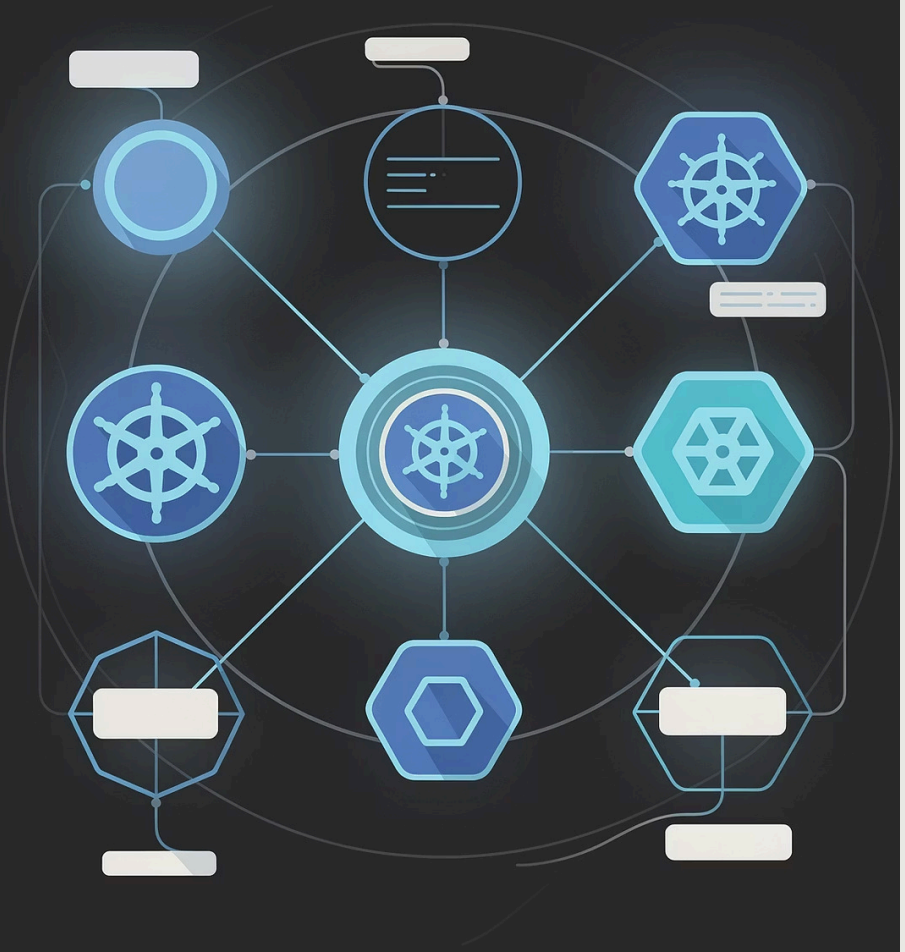


KUBECTL COMMAND REFERENCE

The ultimate CKA exam cheat sheet — every essential kubectl command organized by category, with exam tips, speed tricks, and troubleshooting patterns. Bookmark this before your next cluster session.

CKA EXAM PREP

KUBECTL REFERENCE

WHAT'S INSIDE THIS REFERENCE

This cheat sheet covers all major kubectl command categories tested on the CKA exam. Each section is organized for quick lookup — whether you're mid-exam or administering a live cluster.

01

CLUSTER & NODE OPS

Cluster info, node inspection, API resources, component status

02

WORKLOADS

Pods, Deployments, ReplicaSets — create, scale, rollback

03

NETWORKING & STORAGE

Services, namespaces, PVs, PVCs, port-forwarding

04

CONFIG & SECRETS

ConfigMaps, Secrets, labels, taints, context management

05

DEBUG & SPEED

Logs, events, exec, aliases, YAML generation tricks

CLUSTER INFORMATION COMMANDS

Start every CKA task by orienting yourself — confirm which cluster you're on and verify its health before making any changes.

Command	What It Does
`kubectl cluster-info`	Show control plane and CoreDNS endpoint URLs
`kubectl get nodes`	List all nodes and their Ready/NotReady status
`kubectl get nodes -o wide`	Add IPs, OS, container runtime, and kernel version
`kubectl describe node <name>`	Full node details: taints, conditions, allocated resources
`kubectl get cs`	Component status (etcd, scheduler, controller-manager) — note: deprecated in v1.19+
`kubectl api-resources`	List all available resource types and their short names
`kubectl api-versions`	List all supported API group versions in the cluster
`kubectl get all`	Show pods, services, deployments, replicasets in current namespace

Exam Tip: Always check `kubectl config get-contexts` first — CKA tasks specify which cluster to use. Switching to the wrong context is a common mistake.

CHAPTER 1

CONTEXT & KUBECONFIG MANAGEMENT

The CKA exam runs across **multiple clusters**. You must switch contexts before each task — this is one of the most critical skills to get right under time pressure.

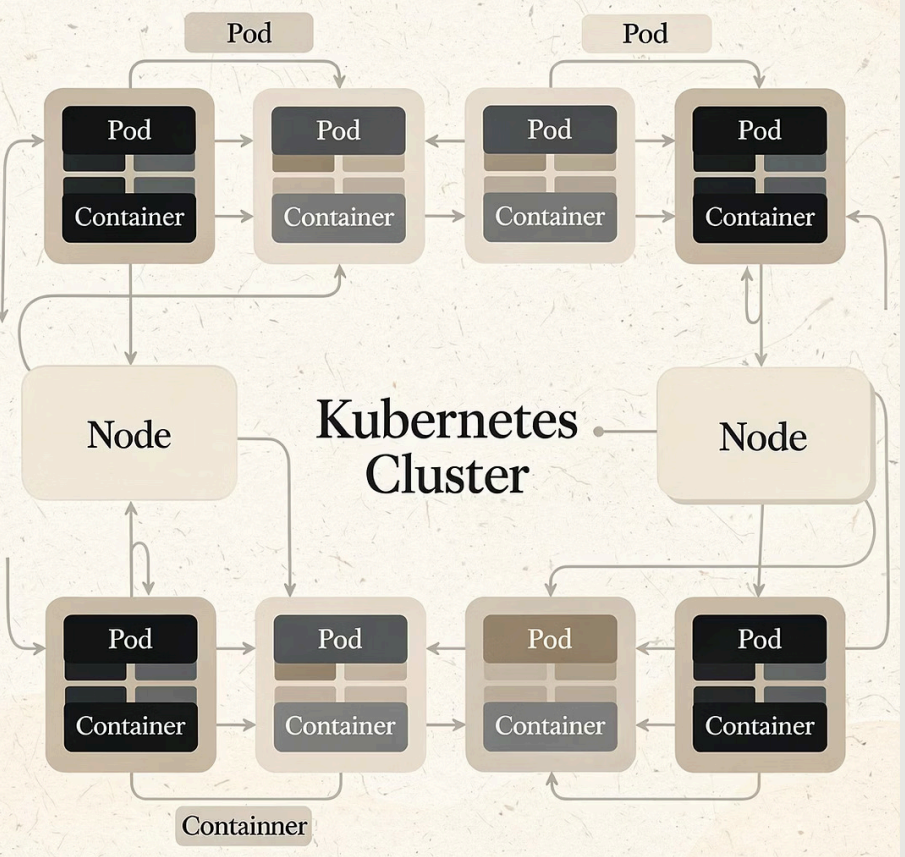
Command	What It Does
`kubectl config get-contexts`	List all contexts with current context marked by *
`kubectl config use-context <name>`	Switch to a different cluster context
`kubectl config view`	Display full kubeconfig (clusters, users, contexts)
`kubectl config current-context`	Print the name of the active context
`kubectl config set-context --current --namespace=dev`	Set default namespace for current context

⚡ EXAM WORKFLOW

Each CKA task tells you which context to use. Run `kubectl config use-context <name>` before anything else — it's the first line in every official task prompt.

WHY IT MATTERS

Working in the wrong cluster wastes time and earns zero points. Make context-switching muscle memory before exam day. Practice it in every lab session.



CHAPTER 2

POD COMMANDS

Pods are the atomic unit of Kubernetes. The CKA exam heavily tests imperative pod creation, inspection, and deletion — master these cold.

POD COMMANDS — FULL REFERENCE

Command	What It Does
`kubectl run nginx --image=nginx`	Create a pod imperatively (no deployment)
`kubectl run busybox --image=busybox -- sleep 3600`	Create pod with a specific startup command
`kubectl run nginx --image=nginx --labels="env=prod"`	Create pod with labels applied at creation
`kubectl get pods`	List pods in current namespace
`kubectl get pods -o wide`	Show node assignment, pod IP, and nominated node
`kubectl get pods -w`	Watch pod status changes live (Ctrl+C to stop)
`kubectl describe pod <name>`	Full pod details: events, conditions, container status
`kubectl get pod <name> -o yaml`	Export full pod spec as YAML
`kubectl get pod <name> -o json`	Export full pod spec as JSON
`kubectl edit pod <name>`	Open pod spec in default editor (limited fields editable)
`kubectl delete pod <name>`	Delete a single pod
`kubectl delete pods --all`	Delete all pods in current namespace
`kubectl delete pod,svc nginx`	Delete multiple resource types by name at once

YAML GENERATION — EXAM SUPERPOWER

Never write YAML from scratch on the exam. Use `--dry-run=client -o yaml` to generate a template, redirect it to a file, edit it, then apply. This is the fastest workflow for complex tasks.

Command	What It Generates
`kubectl run nginx --image=nginx --dry-run=client -o yaml`	Pod YAML template (printed to stdout)
`kubectl run nginx --image=nginx --dry-run=client -o yaml > pod.yaml`	Save pod YAML to file for editing
`kubectl create deployment nginx --image=nginx --dry-run=client -o yaml`	Deployment YAML template
`kubectl create deployment nginx --image=nginx --dry-run=client -o yaml > deploy.yaml`	Save deployment YAML to file
`kubectl apply -f file.yaml`	Apply YAML (creates or updates resource)
`kubectl create -f file.yaml`	Create resource from YAML (fails if exists)
`kubectl delete -f file.yaml`	Delete all resources defined in the YAML file

Pro Workflow: Generate → Edit → Apply. Example: kubectl run pod1 --image=nginx --dry-run=client -o yaml > pod.yaml, then add resources:, env:, or volumeMounts: fields, then kubectl apply -f pod.yaml.

DEPLOYMENT COMMANDS

Deployments manage rollouts and self-healing. The CKA exam tests scaling, image updates, rollbacks, and status checks — know all of these by heart.

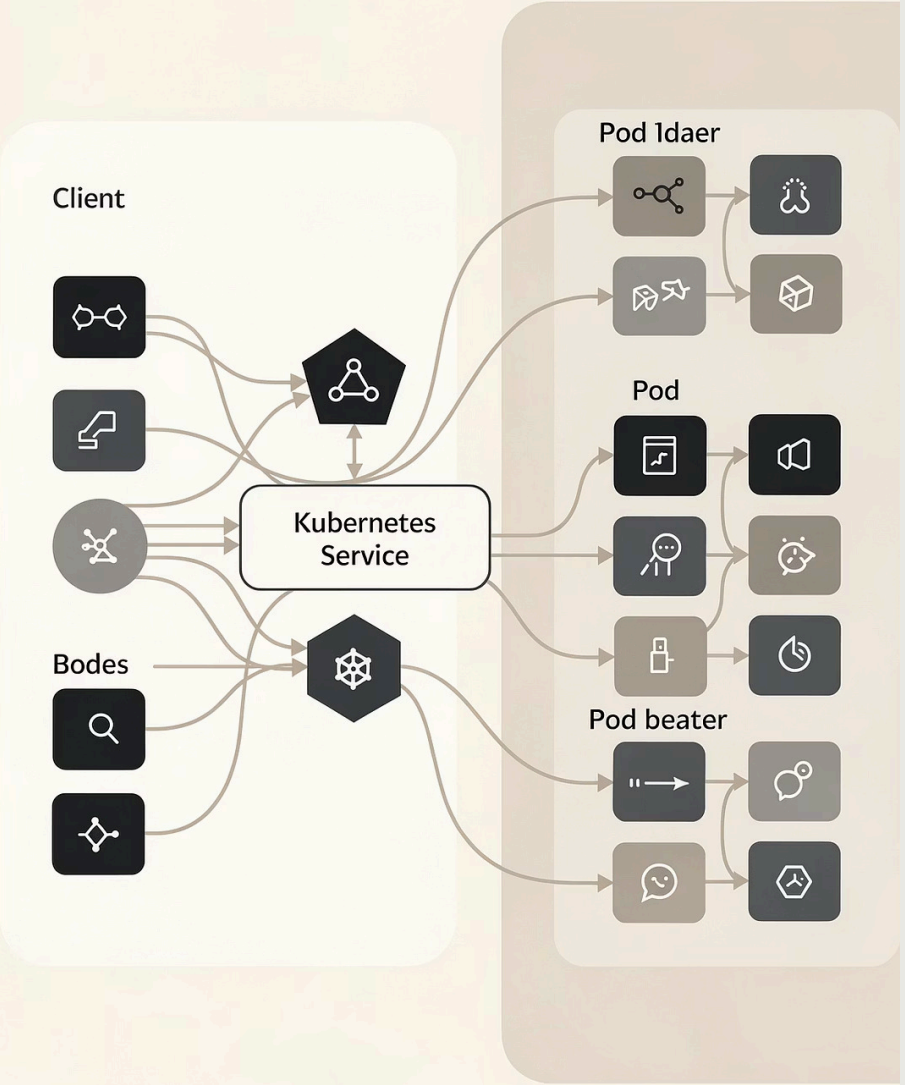
Command	What It Does
`kubectl create deployment nginx --image=nginx`	Create deployment imperatively
`kubectl get deployments`	List all deployments with ready/available counts
`kubectl describe deployment nginx`	Full deployment details including strategy and events
`kubectl scale deployment nginx --replicas=3`	Scale deployment to 3 replicas immediately
`kubectl set image deployment/nginx nginx=nginx:1.25`	Rolling update to new container image
`kubectl rollout status deployment nginx`	Watch rollout progress until complete
`kubectl rollout history deployment nginx`	View revision history with change causes
`kubectl rollout undo deployment nginx`	Rollback to previous revision
`kubectl rollout restart deployment nginx`	Trigger rolling restart (recreates all pods)
`kubectl patch deployment nginx -p '{"spec":{"replicas":5}}'`	Inline JSON patch — fast config change without editing
`kubectl delete deployment nginx`	Delete deployment and all its pods

REPLICASET COMMANDS

ReplicaSets are usually managed by Deployments, but the CKA exam may ask you to inspect or delete them directly — especially during rollback scenarios.

Command	What It Does
`kubectl get rs`	List all ReplicaSets (shows desired vs. ready counts)
`kubectl describe rs <name>`	Full ReplicaSet details including selector and pod template
`kubectl delete rs <name>`	Delete a ReplicaSet (Deployment will recreate it)

Key Insight: When you roll back a Deployment, Kubernetes scales down the current ReplicaSet and scales up the previous one. Old ReplicaSets are kept (with 0 replicas) to enable rollback. Use `kubectl get rs` to see this in action.



CHAPTER 4

SERVICE COMMANDS

Services expose pods to traffic — either internally (ClusterIP), on a node port (NodePort), or externally (LoadBalancer). Knowing how to create and inspect services imperatively is a core CKA skill.

SERVICE COMMANDS — FULL REFERENCE

Command	What It Does
`kubectl expose deployment nginx --port=80 --type=ClusterIP`	Internal-only ClusterIP service (default type)
`kubectl expose deployment nginx --type=NodePort --port=80`	NodePort service — accessible on each node's IP
`kubectl expose deployment nginx --type=LoadBalancer --port=80`	External LoadBalancer (cloud provider provisioned)
`kubectl get svc`	List all services with types, cluster IPs, and ports
`kubectl describe svc nginx`	Full service details including Endpoints (pod IPs)
`kubectl port-forward pod/nginx 8080:80`	Forward local port 8080 to pod port 80 (testing)
`kubectl port-forward svc/nginx 8080:80`	Forward local port 8080 to service port 80
`kubectl delete svc nginx`	Delete the service (pods continue running)

NAMESPACE COMMANDS

Namespaces provide logical isolation within a cluster. CKA tasks frequently specify a namespace — always pass `-n <namespace>` or set the default context namespace to avoid working in the wrong scope.

Command	What It Does
`kubectl get ns`	List all namespaces (short form of `get namespaces`)
`kubectl create namespace dev`	Create a new namespace imperatively
`kubectl delete namespace dev`	Delete namespace and all its resources
`kubectl run nginx --image=nginx -n dev`	Create pod in the specified namespace
`kubectl get pods -n dev`	List pods in the specified namespace
`kubectl get pods --all-namespaces`	List pods across all namespaces (also: `-A`)
`kubectl config set-context --current --namespace=dev`	Change default namespace for current context

LOGS, EXEC & DEBUGGING

When a pod misbehaves, these commands are your first line of investigation. Learn `logs`, `exec`, and `describe` patterns before exam day — they cover 80% of debugging scenarios.

Command	What It Does
`kubectl logs <pod>`	Print current logs from the pod's main container
`kubectl logs <pod> -c <container>`	Logs from a specific container in a multi-container pod
`kubectl logs <pod> --previous`	Logs from the previously crashed container instance
`kubectl logs <pod> -f`	Follow/stream logs in real time (Ctrl+C to stop)
`kubectl exec -it <pod> -- /bin/bash`	Open interactive shell inside running container
`kubectl exec <pod> -- ls /etc`	Run a single command inside container (non-interactive)
`kubectl cp file.txt nginx:/tmp`	Copy local file into pod at specified path
`kubectl cp nginx:/tmp/file.txt .`	Copy file from pod to local current directory
`kubectl get events --sort-by=.metadata.creationTimestamp`	View events sorted by time — great for diagnosing failures
`kubectl describe pod <name>`	Show events, conditions, restart counts, exit codes

CHAPTER 7

CONFIGMAP COMMANDS

ConfigMaps decouple configuration from container images. They're frequently referenced in CKA tasks involving environment variables and volume mounts — know how to create and inspect them imperatively.

Command	What It Does
`kubectl create configmap app-config --from-literal=env=prod`	Create ConfigMap from a key-value literal
`kubectl create configmap app-config --from-file=config.txt`	Create ConfigMap from a file (key = filename)
`kubectl get configmaps`	List all ConfigMaps in current namespace
`kubectl describe configmap app-config`	Show ConfigMap data and metadata
`kubectl edit configmap app-config`	Directly edit ConfigMap data in-place
`kubectl get configmap app-config -o yaml`	Export ConfigMap as YAML for inspection or backup

SECRET COMMANDS

Secrets store sensitive data like passwords, tokens, and keys. They're base64-encoded (not encrypted by default). CKA tasks may ask you to create Secrets and reference them in pod specs via env vars or volume mounts.

Command	What It Does
`kubectl create secret generic db-secret --from-literal=password=admin`	Create generic secret from a literal key-value
`kubectl create secret generic tls-secret --from-file=key.pem`	Create secret from a file (key = filename)
`kubectl create secret generic my-secret --from-file=username.txt`	Create secret using a text file as source
`kubectl get secrets`	List all secrets (values are hidden)
`kubectl describe secret db-secret`	Show secret metadata and key names (not values)
`kubectl get secret db-secret -o yaml`	Show base64-encoded values for decoding

Decode a Secret Value: `kubectl get secret db-secret -o jsonpath='{.data.password}' | base64 --decode` — this is a common exam task pattern.

CHAPTER 9

STORAGE COMMANDS

PersistentVolumes (PV) and PersistentVolumeClaims (PVC) provide durable storage to pods. CKA tasks often ask you to bind a PVC to a PV and mount it in a pod — know how to inspect and troubleshoot binding issues.

Command	What It Does
`kubectl get pv`	List all PersistentVolumes (cluster-scoped)
`kubectl get pvc`	List PersistentVolumeClaims in current namespace
`kubectl describe pvc mypvc`	Show binding status, access modes, and events
`kubectl delete pvc mypvc`	Delete PVC (PV release policy determines what happens to data)
`kubectl get pv -o wide`	Show reclaim policy, access modes, and storage class

NODE MANAGEMENT COMMANDS

Node management is a core CKA topic. Draining, cordoning, and uncordoning nodes are tested directly — you must know the difference between cordon (no new pods) and drain (evict existing pods too).

Command	What It Does
`kubectl cordon <node>`	Mark node as unschedulable — no new pods will be placed here
`kubectl drain <node> --ignore-daemonsets`	Evict all pods from node (except DaemonSets) before maintenance
`kubectl drain <node> --ignore-daemonsets --force`	Force drain — also evicts pods not managed by a controller
`kubectl uncordon <node>`	Re-enable scheduling on a previously cordoned/drained node
`kubectl label nodes node1 disktype=ssd`	Add a label to a node (used with nodeSelector)
`kubectl taint nodes node1 key=value:NoSchedule`	Add a taint — pods without matching toleration won't schedule here
`kubectl taint nodes node1 key-`	Remove a taint from a node (note the trailing dash)
`kubectl describe node <name>`	View taints, conditions, capacity, and allocated resources
`kubectl describe node | grep Allocated`	Quick view of resource allocation across the node

Drain vs. Cordon: Use `cordon` to prevent new scheduling without disrupting running pods. Use `drain` before node maintenance — it cordons AND evicts. Always `uncordon` after maintenance is complete.

CHAPTER 10

TAINTS & TOLERATIONS — DEEP DIVE

Taints repel pods from nodes; tolerations allow specific pods to bypass taints. This pairing is a common CKA scenario — you may be asked to schedule a pod onto a tainted node.

TAINT EFFECTS

- **NoSchedule** — New pods without toleration won't be scheduled
- **PreferNoSchedule** — Scheduler avoids the node but may still use it
- **NoExecute** — Evicts existing pods that don't tolerate the taint

TOLERATION IN POD SPEC

```
tolerations:
- key: "key"
  operator: "Equal"
  value: "value"
  effect: "NoSchedule"
```

LABEL COMMANDS

Labels are the backbone of Kubernetes selectors. Services, deployments, and network policies all rely on label matching. The CKA exam tests adding labels, filtering by them, and removing them imperatively.

Command	What It Does
`kubectl label pod nginx env=prod`	Add or update a label on a pod
`kubectl label pod nginx env-`	Remove a label from a pod (trailing dash removes it)
`kubectl get pods --show-labels`	List pods with all their labels displayed
`kubectl get pods -l env=prod`	Filter pods matching a specific label selector
`kubectl get pods -l 'env in (prod,staging)'`	Set-based label selector with multiple values
`kubectl label nodes node1 disktype=ssd`	Label a node — used for nodeSelector scheduling
`kubectl get nodes --show-labels`	List nodes with all labels (useful for nodeSelector tasks)

OUTPUT FORMATTING COMMANDS

Output formatting is a hidden gem in the CKA exam. Custom columns and JSONPath let you extract exactly the field you need without grepping through walls of text. Practice these before exam day.

Command	What It Does
`kubectl get pod nginx -o yaml`	Full pod spec in YAML format
`kubectl get pod nginx -o json`	Full pod spec in JSON format
`kubectl get pods -o custom-columns=NAME:.metadata.name,IMAGE:.spec.containers[*].image`	Custom column output — show only name and image
`kubectl get pods -o jsonpath='{.items[*].metadata.name}'`	Extract a specific field using JSONPath expression
`kubectl get nodes -o jsonpath='{.items[*].status.addresses[?(@.type=="InternalIP")].address}'`	Extract internal IPs of all nodes
`kubectl get pods --sort-by=.metadata.creationTimestamp`	Sort pods by creation time (oldest first)
`kubectl get events --sort-by=.metadata.creationTimestamp`	Sort events chronologically for debugging

Exam Tip: JSONPath is tested in CKA. A typical task: "Store the internal IP of node01 into /opt/node-ip.txt." Use `kubectl get node node01 -o jsonpath='{.status.addresses[0].address}' > /opt/node-ip.txt`.

CHAPTER 13

MONITORING & RESOURCE METRICS

The `kubectl top` command requires the Metrics Server to be installed. On the CKA exam, it is pre-installed. Use it to identify resource-hungry pods and nodes quickly during troubleshooting tasks.

Command	What It Does
`kubectl top nodes`	Show CPU and memory usage per node in real time
`kubectl top pods`	Show CPU and memory usage per pod in current namespace
`kubectl top pods --all-namespaces`	Pod resource usage across all namespaces
`kubectl top pods --sort-by=cpu`	Sort pods by CPU consumption (highest first)
`kubectl top pods --sort-by=memory`	Sort pods by memory consumption (highest first)
`kubectl describe node <name> | grep -A5 "Allocated"`	View requested vs. allocatable resources on a node

TROUBLESHOOTING WORKFLOW

When something is broken in the CKA exam, follow a systematic approach. These commands form the standard diagnostic sequence — start broad with events, then narrow down to the specific resource.

1

1. CHECK EVENTS

`kubectl get events --sort-by=.metadata.creationTimestamp`

Look for Warning events — they point directly to the failure

2

2. DESCRIBE RESOURCE

`kubectl describe pod <name>`

Check Conditions, Exit Codes, and the Events section at the bottom

3

3. CHECK LOGS

`kubectl logs <pod> --previous`

Previous logs show why a container crashed before restart

4

4. EXEC & INSPECT

`kubectl exec -it <pod> -- /bin/sh`

Get inside the container and verify config, connectivity, files

For cluster-level failures, also check `kubectl get cs`, `kubectl get nodes`, and static pod manifests in `/etc/kubernetes/manifests/` on the control plane node.

THE EXPLAIN COMMAND — BUILT-IN DOCS

`kubectl explain` is your offline documentation during the exam. No internet, no man pages — but `explain` gives you field definitions and valid values for any resource spec, right in the terminal.

Command	What It Does
`kubectl explain pod`	Top-level fields for the Pod resource
`kubectl explain pod.spec`	All fields under pod.spec with types and descriptions
`kubectl explain pod.spec.containers`	Container spec fields — image, ports, env, resources, etc.
`kubectl explain deployment.spec.template.spec`	Deep-dive into deployment pod template spec
`kubectl explain pvc.spec`	PVC fields — accessModes, storageClassName, resources
`kubectl explain --recursive pod.spec`	Recursively list all nested fields (great for discovery)

Exam Strategy: Forgot the exact field name for `tolerations` or `nodeAffinity`? Don't guess — run `kubectl explain pod.spec --recursive | grep -i affinity` to find it instantly without leaving the terminal.

SPEED TIPS

⚡ CKA SPEED TRICKS

The CKA exam is time-constrained — 2 hours for ~17 tasks. Every second saved compounds. These shell tricks can save you 10–15 minutes over the course of the exam.

SPEED TIPS

ALIASES & SHELL SETUP

Set these up at the very start of the exam before touching any task. They apply for the entire session and dramatically reduce keystrokes on every command.

ESSENTIAL ALIASES

```
alias k=kubectl
alias kg='kubectl get'
alias kd='kubectl describe'
alias kdel='kubectl delete'
alias kaf='kubectl apply -f'
export do='--dry-run=client -o yaml'
```

HOW TO USE THEM

```
# Instead of long commands:
kubectl get pods -o wide
kubectl describe pod nginx
kubectl apply -f pod.yaml

# Use short aliases:
kg pods -o wide
kd pod nginx
kaf pod.yaml

# YAML generation shortcut:
k run nginx --image=nginx $do
```

Enable kubectl autocomplete: source <(kubectl completion bash) and complete -F __start_kubectl k — this gives tab-completion for resource names, flags, and subcommands even with the k alias.

SPEED TIPS

SHORT RESOURCE NAMES CHEAT SHEET

Every frequently used resource has a short name. Using short names saves keystrokes and reduces typos — especially important under exam pressure.

WORKLOADS

- `po` → pods
- `deploy` → deployments
- `rs` → replicasets
- `ds` → daemonsets
- `sts` → statefulsets
- `job` → jobs
- `cj` → cronjobs

NETWORKING

- `svc` → services
- `ep` → endpoints
- `ing` → ingresses
- `netpol` → networkpolicies

CONFIG & STORAGE

- `cm` → configmaps
- `secret` → secrets
- `pv` → persistentvolumes
- `pvc` → persistentvolumeclaims
- `sc` → storageclasses

RBAC & CLUSTER

- `ns` → namespaces
- `no` → nodes
- `sa` → serviceaccounts
- `clusterrole` → clusterroles
- `crb` → clusterrolebindings
- `rb` → rolebindings

RBAC COMMANDS

Role-Based Access Control (RBAC) is a dedicated CKA domain. You'll be asked to create Roles, ClusterRoles, and bind them to users or service accounts. Know the imperative commands — they're much faster than writing YAML.

Command	What It Does
`kubectl create role pod-reader --verb=get,list --resource=pods`	Create Role with specific verbs and resources
`kubectl create rolebinding read-pods --role=pod-reader --user=jane`	Bind Role to a user in current namespace
`kubectl create clusterrole cluster-reader --verb=get,list --resource=pods`	Create cluster-wide Role (all namespaces)
`kubectl create clusterrolebinding read-all --clusterrole=cluster-reader --user=jane`	Bind ClusterRole to user across all namespaces
`kubectl auth can-i get pods --as=jane`	Test whether a user has permission to perform an action
`kubectl auth can-i get pods --as=jane -n dev`	Test permissions in a specific namespace
`kubectl get roles`	List Roles in current namespace
`kubectl get clusterroles`	List all ClusterRoles in the cluster

SERVICEACCOUNT COMMANDS

ServiceAccounts provide an identity for pods to authenticate to the Kubernetes API. CKA tasks often ask you to create a ServiceAccount and bind a Role to it — then assign it to a pod via `serviceAccountName`.

Command	What It Does
`kubectl create serviceaccount my-sa`	Create a ServiceAccount in current namespace
`kubectl get serviceaccounts`	List all ServiceAccounts (short: `kubectl get sa`)
`kubectl describe sa my-sa`	Show ServiceAccount details including mounted secrets
`kubectl create rolebinding sa-bind --role=pod-reader --serviceaccount=default:my-sa`	Bind Role to a ServiceAccount (format: namespace:name)

Pod Spec Reference: To assign a ServiceAccount to a pod, add `serviceAccountName: my-sa` under `spec:` in the pod YAML. Use `kubectl explain pod.spec.serviceAccountName` during the exam if you forget the exact field name.

EXAM DAY

CKA EXAM DAY CHECKLIST

Preparation meets execution. These habits separate candidates who pass from those who run out of time — follow this routine at the start of every exam session.

EXAM DAY

START-OF-EXAM SETUP ROUTINE

Run this setup sequence before touching any task. Takes under 60 seconds and pays dividends for the full 2-hour exam.

1 SET ALIASES IMMEDIATELY

Run `alias k=kubectl` and `export do='--dry-run=client -o yaml'` – these save keystrokes on every single command for the rest of the exam.

2 ENABLE TAB COMPLETION

Run `source <(kubectl completion bash)` and `complete -F __start_kubectl k` – tab-complete resource names and flags to avoid typos under pressure.

3 READ EACH TASK FULLY

Note the context name, namespace, and output file path (if any) before typing a single command. Missing the namespace is the #1 source of lost points.

4 SWITCH CONTEXT FIRST

Run `kubectl config use-context <name>` before every task. Verify with `kubectl config current-context` – never assume you're in the right cluster.

5 FLAG HARD TASKS & MOVE ON

If a task takes more than 5 minutes, flag it and skip. Return at the end. Easier high-value tasks later in the exam are worth more than getting stuck on one hard problem.

COMPLETE COMMAND QUICK REFERENCE

The master table — every command from this reference in one place. Use this as your final review before the exam.

🔵 CLUSTER & NODES

- k cluster-info
- k get nodes -o wide
- k describe node <n>
- k cordon / drain / uncordon
- k taint nodes <n> key=val:NoSchedule
- k top nodes

🟢 PODS & WORKLOADS

- k run nginx --image=nginx
- k get pods -o wide -w
- k describe pod / logs / exec
- k create deploy nginx --image=nginx
- k scale deploy nginx --replicas=3
- k rollout undo / restart deploy

🟡 CONFIG & SECRETS

- k create cm app --from-literal=k=v
- k create secret generic s --from-literal=p=v
- k get cm / secrets -o yaml
- k create sa my-sa
- k create role / rolebinding
- k auth can-i get pods --as=user

🔴 DEBUG & SPEED

- k get events --sort-by=.metadata.creationTimestamp
- k explain pod.spec --recursive
- k run nginx $do > pod.yaml
- k config use-context <name>
- k get all -n <ns>
- k cp / port-forward

YOU'RE READY. GO PASS THE CKA. 🎯

This reference covers every kubectl command tested on the Certified Kubernetes Administrator exam — organized, complete, and exam-proven. The commands are the easy part once they're in muscle memory. Trust your preparation, read each task carefully, manage your time, and remember: **explain, dry-run, apply** is the workflow that wins.

17

EXAM TASKS

Approximate number of tasks across multiple clusters

2H

TIME LIMIT

120 minutes — roughly 7 minutes per task on average

66%

PASSING SCORE

Minimum score required to earn the CKA certification

1YR

EXAM RETAKE

One free retake included with your exam registration

Final Reminder: The exam environment has kubectl docs and the Kubernetes documentation at kubernetes.io/docs available. Use it — especially for complex YAML structures like NetworkPolicy, PersistentVolumes, and RBAC. Good luck! 🚀